Dispensa Operativa di Laboratorio per Chase

Progetto: Automa Meteo in Python, Storico Dati e Git Integration
Guida completa passo-passo per le attività di laboratorio

Benvenuto Chase!
Questa dispensa guida passo dopo passo nella creazione del tuo primo progetto di acquisizione dati in Python. L'obiettivo è realizzare un **automa software** che acquisisce la temperatura reale in tempo reale tramite un'API REST, la registra in un database CSV aperto ed elabora una Dashboard grafica web aggiornata automaticamente. Tutte le versioni del codice saranno salvate e tracciate su un repository Git condiviso.

1. Obiettivi e Architettura del Progetto

L'applicazione simula il funzionamento di un sistema di controllo industriale distribuito (come EPICS). L'architettura è suddivisa nei quattro blocchi fondamentali descritti nella seguente tabella:

Componente	Ruolo nel Sistema	Tecnologia e Tool Utilizzati
1. Sorgente Dati	Fornisce i valori reali di temperatura misurati all'esterno.	API REST pubblica e gratuita (Open-Meteo)
2. Automa Python	Legge la misura ad intervalli regolari, scrive i file e gestisce	OnlineGDB (Compiler Python 3 nel browser)

Componente	Ruolo nel Sistema	Tecnologia e Tool Utilizzati
	la logica.	
3. Database Storico	Memorizza la cronologia delle letture per analisi successive in tabella.	File CSV (database_temperatura.csv) aperto in Excel
4. Dashboard Grafica	Presenta lo stato attuale del sistema in una scheda grafica pulita.	File HTML (dashboard.html) visualizzabile nel browser

2. Gli Strumenti di Lavoro (Tooling)

- ---

  Editor e Ambiente di Esecuzione: OnlineGDB (onlinegdb.com). Non richiede alcuna installazione sul PC scolastico. Si apre direttamente nel browser, selezionando *Python 3* nel menu in alto a destra.
- **Versionamento e Condivisione Codice:** Repository Git condiviso su GitHub/GitLab. Permette di salvare lo storico di tutti i miglioramenti apportati e lavorare in squadra con il docente.
- **Interfaccia Grafica e Report:** Browser web locale (Chrome, Edge o Firefox) per visualizzare la Dashboard HTML generata dall'automa.

3. Fase 1: Configurazione del Repository Git Condiviso

Prima di scrivere il codice, prepariamo lo spazio di lavoro condiviso:

1. Accedi al tuo account su **GitHub / GitLab**.
2. Crea un nuovo repository chiamandolo **automa-meteo-python** (impostalo come *Private* o *Public* a seconda della direttiva del docente).
3. Vai su **Settings > Collaborators** e aggiungi il docente tra i collaboratori con permessi di accesso, così da poterti supportare direttamente sul codice.

4. Aggiungi un file iniziale README.md con la descrizione del progetto.

4. Fase 2: Il Codice Sorgente Completo dell'Automa

Questo è il codice principale dell'automa. Copialo ed incollalo nell'editor centrale di OnlineGDB:

```
import urllib.request
import json
import time

# URL dell'API REST pubblica per la temperatura reale (Coordinate di Milano)
URL_API = "https://api.open-meteo.com/v1/forecast?latitude=45.4642&longitude=9.1900&current_weather=true"

FILE_DATABASE = "database_temperatura.csv"
FILE_DASHBOARD = "dashboard.html"

def leggi_temperatura_api():
    """Connessione all'API REST e lettura della temperatura reale."""
    try:
        req = urllib.request.Request(URL_API, headers={'User-Agent': 'Python-School-App'})
        with urllib.request.urlopen(req) as response:
            data = json.loads(response.read().decode())
            temp = data["current_weather"]["temperature"]
            return temp
    except Exception as e:
        print(f"Errore durante il recupero del dato: {e}")
        return None

def salva_nel_database(ora, data, temp):
    """Registrazione del dato nel file CSV (compatibile con Excel)."""
    try:
        with open(FILE_DATABASE, "x") as f:
            f.write("Data;Ora;Temperatura_C\n")
    except FileExistsError:
        pass

    with open(FILE_DATABASE, "a") as f:
        f.write(f"{data};{ora};{temp}\n")

def genera_dashboard(ora, data, temp):
    """Generazione dell'interfaccia grafica web HTML."""
    html_content = f"""


    <meta http-equiv="refresh" content="10">
    <title>Pannello Monitoraggio Temperatura</title>
    <style>
```

```
        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #1e1e2e;
            color: #cdd6f4;
            text-align: center;
            padding-top: 40px;
        }}
        .card {{
            background-color: #313244;
            display: inline-block;
            padding: 30px 50px;
            border-radius: 16px;
            box-shadow: 0 10px 25px rgba(0,0,0,0.5);
            border: 2px solid #45475a;
        }}
        .valore {{
            font-size: 64px;
            font-weight: bold;
            color: #89b4fa;
            margin: 20px 0;
        }}
        .status {{
            color: #a6e3a1;
            font-size: 14px;
            font-weight: bold;
        }}
    </style>
</head>
<body>
    <div class="card">
        <h2>STAZIONE METEO - MONITORAGGIO REALE</h2>
        <p>Ultimo Aggiornamento: <b>{data}</b> alle <b>{ora}</b></p>
        <div class="valore">{temp} °C</div>
        <p class="status">● Automa attivo - Connessione API REST OK</p>
        <p><small>Dati salvati in continuo nel database: <i>{FILE_DATABASE}</i></small></p>
    </div>
</body>
</html>"""

    with open(FILE_DASHBOARD, "w", encoding="utf-8") as f:
        f.write(html_content)

# --- CICLO DI ESECUZIONE CONTINUA ---
print("🚀 Automa avviato su OnlineGDB!")

while True:
    ora_attuale = time.strftime("%H:%M:%S")
    data_attuale = time.strftime("%d/%m/%Y")

    print(f"\n[{ora_attuale}] Interrogazione API REST in corso...")
```

```
    temperatura = leggi_temperatura_api()

    if temperatura is not None:
        print(f"-> Temperatura attuale: {temperatura} °C")
        salva_nel_database(ora_attuale, data_attuale, temperatura)
        print(f"-> Misurazione registrata su '{FILE_DATABASE}'")
        genera_dashboard(ora_attuale, data_attuale, temperatura)
        print(f"-> Dashboard '{FILE_DASHBOARD}' aggiornata!")
    else:
        print("-> Impossibile recuperare il dato dall'API.")

    # Attesa di 60 secondi prima del prossimo ciclo
    time.sleep(60)
```

5. Fase 3: Guida Operativa di Esecuzione in Laboratorio

1. ---

   Avvio dell'Automa: Clicca sul pulsante verde **Run** in alto a sinistra su OnlineGDB. Nella console nera in basso vedrai scorrere i log dell'automa ogni 60 secondi.
2. **Verifica dei File Generati:** Nella barra dei file a sinistra su OnlineGDB vedrai comparire database_temperatura.csv e dashboard.html.
3. **Visualizzazione della Dashboard:** Scarica sul PC il file dashboard.html e fai doppio clic per vederlo nel browser: la scheda si ricaricherà ogni 10 secondi per mostrare l'ultimo valore disponibile.

6. Fase 4: Commit e Versionamento delle Modifiche su Git

Ad ogni completamento di una funzionalità o alla fine della sessione di laboratorio, salva il lavoro su Git seguendo questa procedura:

1. Nel tuo repository Git su GitHub/GitLab, seleziona **Add File > Create new file** oppure **Upload files**.
2. Nome del file principale: automa_meteo.py.
3. Incolla il codice aggiornato da OnlineGDB.
4. Compila il campo **Commit message** con una spiegazione sintetica del lavoro svolto (es. v1.0: prima versione funzionante dell'automa con API e CSV).
5. Conferma cliccando su **Commit changes**. In questo modo il codice sarà immediatamente visibile anche al docente!